

Lesson 11 : Intro to GUI with Tkinter

Description

Students learn what a graphical user interface is by using Tkinter. In this lesson they will learn how to create a window, label, button and how to show them into window screen. Students will create a few simple projects to practice what they have learned in this lesson and one big project by implementing all of the concepts from the material.

Objectives

1. Understand what GUI application is.
2. Learn the basics of Tkinter framework and to create a GUI window.
3. Practice creating basic GUI elements such as labels and buttons.
4. Implement various placement methods to show GUI elements on the window.
5. Applying button event to handle button clicking.
6. Create a project that implements all of the concepts learned.

Tools

1. Pycharm

Class Structure

Duration	Topic
5 mins	Preparation
5 mins	Introduction to GUI
5 mins	Creating Window
10 mins	Creating Labels
25 mins	Geometry Management
5 mins	Creating Buttons
5 mins	Button Events
25 mins	Project
5 mins	Wrap Up

Planned Activities	Resources
Preparation (5 mins)	
- Teacher greets students and ask how's their conditions. - Instruct students to open their pycharm. - If possible, teacher can incorporate a mini game during this preparation time to engage with the students.	
Introduction to GUI (5 mins)	
Explain – GUI Type of Application - Tell students they are going to learn about GUI for Python. - Ask if students know about GUI or not, or ask them to give us some examples. - Explain that GUI is Graphical User Interface. It is an application which helps us interact with the computer more comfortably. Tell them that even text editor they are using is a GUI app, the games they might play are GUI apps, the browser they are using is a GUI app. - Teacher clicks the links provided and tells students about some of the most popular websites or applications created using Python, such as: Instagram, Pinterest, Google, Spotify, Dropbox, Netflix, Reddit. - Ask the students whether or not they know what GUI is now, and how beautiful it is knowing Instagram uses Python. - Summarise: GUI can make our programs look more aesthetically pleasing compared to just texts.	Website: 10 Applications Built Using Python

Creating Window (5 mins)

Show and Explain – Tkinter Library

- Tell students for this project; we will be using Tkinter. It is a module in Python to create simple GUI app. It is already built-in, so they do not have to worry about installing anything.
- Show the project to the students and ask whether or not they know what they are looking at.
- Explain that they are looking at a window. Furthermore, to create a window, the students will need to import the Tkinter library. Then to use the methods from the library, they would need to create a variable first.
- Teacher also shows that the window can be named and given dimensions. Also, the window will stay open until we close the program.

Code – Creating Window

- First, the students will need to import the Tkinter library.
- Then they need to declare a variable to place the window. They can name the variable “**window**”.
- After creating the window, they can give the window a title, and also specify the dimensions:
 - Giving title to the window: **window.title("Window Name")**
 - Specifying window dimension: **window.geometry("height x width")**
- Lastly, they need to execute **window.mainloop()** to keep the window opened.

File/Code:

[1.CreatingWindow.py](#)

Creating Labels (10 mins)

Show and Explain – Label Widget

- Tell the students that after creating the window, they can create a label inside the window. A label is typically used to display text.

- Show the project to the students and tell them that a label is a widget in Tkinter.
- Explain that to display widgets, the **.pack()** method is needed.

Code – Creating Label

- First, the students need to create a label by using **tkinter.Label**. There are two methods in creating label and show it on the window:
 - First method:
`Label1 = tkinter.Label(window, text="This is label 1")`
`Label1.pack()`
 - Second method:
`tkinter.Label(window, text="This is label 2").pack()`
 - Teachers may recommend the students to use the first method instead because it is more comprehensible.
- For the label with an image in it, we just need a variable to store the image, and then we use the **.PhotoImage()** method. Remember to store the image file in the same folder as the project, or in a subfolder. If the students need to resize the image to fit the label, it is explained in the code file.

File/Code:

[2.CreatingLabel.py](#)

Geometry Management (25 mins)

Explain and Analyze – Placements

- Tell the students that all widgets will have geometry measurements, to help us organize the widgets inside frames or windows.
- Show the projects to the students and show them that there are three different geometry manager classes:
 - **.pack()** method: organize the widgets in stacks before placing them in the parent widget.

- **.grid()** method: organize the widgets in a grid (table-like structure) before placing them in the parent widget.
- **.place()** method: organize the widgets by placing them on specific positions based on x and y coordinates.

Code – Pack Placement

- First, create a label with a red foreground and white background.
- Next, create a label with the background spanning the whole width, using “fill=tkinter.X” inside **.pack()** method: **label.pack(fill=tkinter.X)**
- Tell the students they can create padding using either “padx” to give outer padding on the left and right of the label or “pady” to give padding to the top and bottom of the label.
- There is also internal padding, which uses “ipadx”, or “ipady”.
- Each label they create will stack on top or below each other. To position the label side by side, the students will need to use the “side” option in the **.pack()** method. For example:
label.pack(side=tkinter.LEFT)

Code – Grid Placement

- To use the **.grid()** method, the students need to specify the position of each widget.
- The position is specified using rows and columns.
- The first row and column start with number 0.
- The first row will start from the top side, and the first row will start from the left side.
- For example, if the students want to place a widget on the top left side of the window, they need to type: **label.grid(column=0, row=0)**
- If they want it to be in the next row, they need to increment the row number. Same goes for the columns.
- Then to specify the initial width of the cell, they can use “ipadx”: **label.grid(column=0, row=0, ipadx=30)**
- They can also move the element inside the cell around if the cell is big enough.

File/Code:

[3.PackPlacement.py](#)

File/Code:

[4.GridPlacement.py](#)

- If they want the element to stick to the right side of the cell, they can use “sticky”:
label.grid(column=1, row =2, sticky=”tkinter.E”)
- In the example, “E” is used to represent the East, which means the right side. However, they can also use another letter which represents the other sides. The details are following:
 - N = North = Top
 - S = South = Bottom
 - W = West = Left
 - E = East = Right
- If the students use more than one value, for example, “W” and “E”, the element will stretch itself to the left and right side, filling the whole width of the cell.
- Similar to HTML tables, there are also rowspan, and colspan to extend the size of the cells.

Code – Place Placement

- To use the **.place()** method, they only need to specify the x and y coordinate of the widget, according to the window size. They can also specify the dimension of the element:
label.place(x=30, y=50, width=100, height=50)
- Remind the students of the origin of x and y (0,0) is at the top left of the window.

File/Code:

[5.PlacePlacement.py](#)

Creating Buttons (5 mins)

Show and Explain – Button Widget

- Tell students that after creating the window, the student can create a button inside the window.
- Show the project to the students and tell them that a button is a widget in Tkinter.
- Explain that to display the button, we need to use one of the placement methods. In this case, we will be using **.pack()** method for simplicity.

Code – Creating Button

- First, the students need to create a button using **tkinter.Button(master, option=value)** and save it to a variable.
- Tell the students that parameter “master” is the parent window, “option” is like an attribute in HTML. (i.e. text, height, font).
- Multiple options are separated by commas.
- The way to create a button is similar to how we created labels:
button = tkinter.Button(window, text="This is a button")
button.pack()

File/Code:

[6.CreatingButton.py](#)

Button Events (5 mins)

Show and Explain – Button Event

- Tell students that after creating the button, we can get different outcomes through different events.
- Show the project to the students.
- Explain that events can be key pressed or mouse clicked.

Code – Button Event

- First, the students need to define a function to let the button know what to do when they press it. Remind them to name the function properly.
- Then, they need to use the **.configure()** method. This is to change how the widget will appear on the screen:
def changetext():
label.configure(text="I am a label", fg="red", bg="black")
- After that, they need to create the button as shown before in the previous project.

File/Code:

[7.ButtonEvent.py](#)

- Lastly, they can add a new option to the button, which is “command”. The value for the option should be the function name:

tkinter.Button(window, text="Click Me", command=changetext)

Project (25 mins)

Show and Explain – Project

- Tell students that they are going to create a project by themselves.
- Show the project to the students and remind them of the different outcomes to the events happening.
- Explain that there are several buttons which can be pressed or clicked:
 - Does nothing.
 - Changing the text of a button.
 - Changing the color of a label.
 - Changing the color of a button.
 - Incrementing the counter number.

Code – Project

- First, the students need to define the different functions for the different outcomes.
- Remind the students to initialise the variable for the counter.
- Next, they just need to use what they have learned in this session to put everything together.
- First, of course, is to create the window.
- Then they can place their labels and buttons where they want them. For simplicity, you can tell the students to just use **.pack()** method.
- Next, they just need to add the button events on each elements they want to apply the effect by using command property on the buttons.

File/Code:

[8.Project.py](#)

Wrap Up (5 mins)

Wrapping Up

- We sum up by asking questions, making sure the students remember the most important things.
- We thank students for doing a great job overall.

☐ ☆ Good Luck With The Class !! ☆ ☐